Packages:

- A Package can be defined as a grouping of related types(classes, interfaces)
- A package represents a directory that contains related group of classes and interfaces.
- Packages are used in Java in order to prevent naming conflicts.
- There are two types of packages in Java.
    1. Pre-defined Packages(built-in)
    2. User defined packages

Pre-defined Packages:

Package Name	Description
java.lang	Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.). This package is automatically imported.
java.io	Contains classes for supporting input / output operations.
java.util	Contains utility classes which implement data structures like Linked List, Hash Table, Dictionary, etc and support for Date / Time operations. This package is also called as **Collections.**
java.applet	Contains classes for creating Applets.
java.awt	Contains classes for implementing the components of graphical user interface (like buttons, menus, etc.).
java.net	Contains classes for supporting networking operations.
javax.swing	This package helps to develop GUI Applications. The 'x' in javax represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, javax.swing is an extended package of java.awt.
java.sql	This package helps to connect to databases like Oracle/Sybase/Microsoft Access to perform different operations.

Defining a Package(User defined):

To create a package is quite easy: simply include a **package** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package. The **package** statement defines a name space in which classes are stored. If you omit the **package** statement, the class names are put into the default package, which has no name.

This is the general form of the **package** statement:

package *pkg*;

Here, *pkg* is the name of the package.

For example, the following statement creates a package called **MyPackage**:

package MyPackage;

Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **MyPackage** must be stored in a directory called **MyPackage**. Remember that case is significant, and the directory name must match the package name exactly. More than one file can include the same **package** statement.

Most real-world packages are spread across many files. You can create a hierarchy of packages. To do so, simply separate each package name from the one above it by use of a period. The general form of a multileveled package statement is shown here:

package *pkg1*[.*pkg2*[.*pkg3*]];

A package hierarchy must be reflected in the file system of your Java development system. For example, a package declared as

package java.awt.image;

Example: Package demonstration

```
package pack;
public class Addition
{
        int x,y;
        public Addition(int a, int b)
        {
                x=a;
                y=b;
        }
        public void sum()
        {
                System.out.println("Sum :"+(x+y));
        }
}
```

Step 1: Save the above file with Addition.java

```
package pack;
public class Subtraction
{
```

```
        int x,y;
        public Subtraction(int a, int b)
        {
                x=a;
                y=b;
        }
        public void difference()
        {
                System.out.println("Difference :"+(x-y));
        }
}
```

Step 2: Save the above file with Subtraction.java

Step 3: Compilation

To compile the java files use the following commands

javac -d directory_path name_of_the_java file

Javac –d . name_of_the_java file

Note: -d is a switching option creates a new directory with package name. Directory path represents in which location you want to create package and . (dot) represents current working directory.

```
F:\Courses\JAVA26-27\Unit4\Packages>javac -d . Addition.java

F:\Courses\JAVA26-27\Unit4\Packages>javac -d . Subtraction.java
```



Step 4: Access package from another package

There are three ways to use package in another package:

1. **With fully qualified name.**

```
class UseofPack
{
        public static void main(String arg[])
        {
                pack.Addition a=new  pack.Addition(10,15);
                a.sum();
                pack.Subtraction s=new pack.Subtraction(20,15);
```

```
                s.difference();
        }
}
```

2. **import package.classname;**

```
import pack.Addition;
import pack.Subtraction;
class UseofPack
{
        public static void main(String arg[])
        {
                 Addition a=new  Addition(10,15);
                a.sum();
                Subtraction s=new Subtraction(20,15);
                s.difference();
        }
}
```

3. **import package.*;**

```
import pack.*;
class UseofPack
{
        public static void main(String arg[])
        {
                 Addition a=new  Addition(10,15);
                a.sum();
                Subtraction s=new Subtraction(20,15);
                s.difference();
        }
}
```

Note: Don't place Addition.java, Subtraction.java files parallel to the pack directory. If you place JVM searches for the class files in the current working directory not in the pack directory.

PATH and CLASSPATH environment variables:

PATH

- PATH is an environment variable is used to tell the location of built-in directories such as *javac, java, jar, javadoc, appletviewer* ...
- Path tells the system where to locate the JDK files that contain these commands.

CLASSPATH :

The class path environment variable is used to specify the location of the user defined classes and packages.

How Java Finds a Package (Class Path Resolution):

When a program uses a package, the compiler and JVM must locate the package's .class files. There are **three ways** to tell Java where the package is.

1. Current Working Directory (default): By default, Java starts searching from the current working directory. If the package directory is a subdirectory of the current directory, it is found automatically.

```
F:\Packages\
├── UseofPack.java
└── pack\
    ├── Addition.class
    └── Subtraction.class
```

```
F:\...\Packages > javac UseofPack.java
F:\...\Packages > java UseofPack
```

2. CLASSPATH Environment Variable: Set CLASSPATH to the directory that contains the package directory (the *parent* of pack).

Example: UseofPack.java is in the E drive, and package pack is in the F drive.

```
F:\......\Packages\
└── pack\
    ├── Addition.class
    └── Subtraction.class
E:>
└── UseofPack.java
```

```
E:> set CLASSPATH=F:\......\packages;.;
E: > javac UseofPack.java
E: > java UseofPack
```

```
E:\>set CLASSPATH=F:\Courses\JAVA26-27\Unit4\Packages;.;
```

3. The -cp (or -classpath) Option

```
E:\>javac -cp "F:\Courses\JAVA26-27\Unit4\Packages;.;" UseofPack.java

E:\>java -cp "F:\Courses\JAVA26-27\Unit4\Packages;.;" UseofPack
Sum :25
Difference :5
```

Pass the path directly with the java and javac commands. This applies **only to that command** and does not change any system settings.

Here too, include . when the class you are running is in the current directory.

Access Protection

- Access protection defines actually how much an element (class, method, variable) is exposed to other classes and packages.
- There are four types of access specifiers available in java:
    1. Visible to the class only (private).
    2. Visible to the package (default). No modifiers are needed.
    3. Visible to the package and all subclasses (protected)
    4. Visible to the world (public)

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

- Generally, protected and private are used only in nested classes.

Table 10.1 Access specifiers for classes

Access specifier	*Example of code*	*Access permitted*
No access specifier	`class A {}`	Accessible to classes in same package
public	`public class B{}`	Accessible to all classes in all packages
protected	`public class A{ protected class C {} }`	Used with nested classes; members of host (outer) class can access protected class because it is a member of the class
private	`public class A{ private class C {} }`	Used with nested classes; members of host (outer) class can access private class because it is a member of the class

Example 1:

```
package mypack;

public class A {

   public int publicValue = 10;
   protected int protectedValue = 20;
   int defaultValue = 30;
   private int privateValue = 40;

}
```

Another Package:

```
package testpack;
import mypack.A;
public class B extends A   {
   public static void main(String[] args) {
     B obj = new B();

      System.out.println(obj.publicValue);     //   Allowed

        System.out.println(obj.protectedValue); //   allowed
      // System.out.println(obj.defaultValue);   //   Not allowed
      // System.out.println(obj.privateValue);   //   Not allowed
   }
}
```

Output:

```
F:\Courses\JAVA26-27\Unit4\Packages>javac -d . A.java

F:\Courses\JAVA26-27\Unit4\Packages>javac -d . B.java

F:\Courses\JAVA26-27\Unit4\Packages>java testpack.B
10
20
```

Example 2: From Complete Reference Text Book

The following example shows all combinations of the access control modifiers. This example has two packages and five classes. The source for the first package defines three classes: **Protection**, **Derived**, and **SamePackage**.

Name of the package: pkg1

This file is Protection.java

```
package pkg1;

public class Protection
{
        int n = 1;
        private int n_priv = 2;
        protected int n_prot = 3;
        public int n_publ = 4;

        public Protection()
        {
                System.out.println("base constructor");
                System.out.println("n = " + n);
                System.out.println("n_priv = " + n_priv);
                System.out.println("n_prot = " + n_prot);
                System.out.println("n_publ = " + n_publ);
        }
}
```

This is file Derived.java:

```
package pkg1;

class Derived extends Protection
{
   Derived()
   {
     System.out.println("Same package - derived (from base) constructor");
     System.out.println("n = " + n);

     /* class only
     *  System.out.println("n_priv = "4 + n_priv); */

     System.out.println("n_prot = " + n_prot);
     System.out.println("n_publ = " +n_publ);
   }
}
```

This is file SamePackage.java

```
package pkg1;
```

```
class SamePackage
{
   SamePackage()
   {
     Protection pro = new Protection();
     System.out.println("same package - other constructor");
     System.out.println("n = " + pro.n);

     /* class only
     *  System.out.println("n_priv = " + pro.n_priv); */

     System.out.println("n_prot = " + pro.n_prot);
     System.out.println("n_publ = " + pro.n_publ);
   }
}
```

Name of the package: pkg2

This is file Protection2.java:

```
package pkg2;

class Protection2 extends pkg1.Protection
{
        Protection2()
        {
             System.out.println("Other package-Derived (from Package 1-Base)
          Constructor");

          /* class or package only
          *  System.out.println("n = " + n); */

          /* class only
          *  System.out.println("n_priv = " + n_priv); */

          System.out.println("n_prot = " + n_prot);
          System.out.println("n_publ = " + n_publ);
        }
}
```

This is file **OtherPackage.java**

```
package pkg2;

class OtherPackage
{
   OtherPackage()
```

```
    {
      pkg1.Protection pro = new pkg1.Protection();

      System.out.println("other package - Non sub class constructor");

      /* class or package only
      *  System.out.println("n = " + pro.n); */

      /* class only
      *  System.out.println("n_priv = " + pro.n_priv); */

      /* class, subclass or package only
      *  System.out.println("n_prot = " + pro.n_prot); */

      System.out.println("n_publ = " + pro.n_publ);
    }
}
```

If you want to try these t two packages, here are two test files you can use. The one for package **pkg1** is shown here:

```
/* demo package pkg1 */

package pkg1;

/* instantiate the various classes in pkg1 */
public class Demo
{
   public static void main(String args[])
   {
     Derived obj2 = new Derived();
     SamePackage obj3 = new SamePackage();
   }
}
```

The test file for package pkg2 is

```
package pkg2;

/* instantiate the various classes in pkg2 */
public class Demo2
{
   public static void main(String args[])
   {
     Protection2 obj1 = new Protection2();
```

```
        OtherPackage obj2 = new OtherPackage();
    }
}
```

```
D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>javac -d . Demo.java

D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>javac -d . Demo2.java

D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>java pkg1.Demo
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
Same package - derived (from base) constructor
n = 1
n_prot = 3
n_publ = 4
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
same package - other constructor
n = 1
n_prot = 3
n_publ = 4
```

```
D:\Materials\JAVA Material\Unit 2\Packages\AccessSpecifier>java pkg2.Demo2
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
Other package - Derived (from Package 1-Base)Constructor
n_prot = 3
n_publ = 4
base constructor
n = 1
n_priv = 2
n_prot = 3
n_publ = 4
other package - Non sub class constructor
n_publ = 4
```

java.lang Package and its classes:

- java.lang package is imported into every Java program by default.
- This package contains classes and interfaces that are essential for the design of Java programming language and are as follows:
    - Object class is the root of all classes
    - Primitive data types
    - Classes access system resources and other low level entities
    - Math class – for mathematical operations
    - Class Throwable – for errors and exceptions
    - Thread class – to implement threads
    - To work with strings such as String, StringBuffer

S. No.	Name of the Class	Description
1	Object	Root class of all Java classes.
2	String	Represents a sequence of characters.
3	StringBuffer	Creates mutable and thread-safe character sequences.
4	StringBuilder	Creates mutable character sequences efficiently.
5	Math	Provides mathematical methods such as sqrt(), pow(), and abs().
6	System	Provides standard input/output streams and system-related methods.
7	Class	Represents a class or interface at runtime.
8	Integer	Wrapper class for the primitive type int.
9	Thread	Represents a thread of execution.
10	Exception	Represents conditions that a program may handle using exception-handling mechanisms.

Interfaces in java.lang package:

1. Appendable - Allows characters or character sequences to be appended.
2. AutoCloseable - Allows resources to be closed automatically using try-with-resources.
3. CharSequence - Represents a sequence of characters. Implemented by String, StringBuilder, and StringBuffer.
4. Cloneable - Indicates that a class permits cloning of its objects.

5. Comparable<T> Defines natural ordering using the compareTo(T o) method.
6. Iterable<T> - Allows objects to be used in enhanced for loops.
7. Readable - Represents a source of characters that can be read into a CharBuffer.
8. Runnable - Represents a task that can be executed by a thread.

Class Object:

- The Object class is the root superclass of all classes in Java.
- It belongs to the java.lang package.
- Every class directly or indirectly inherits from Object, except that interfaces do not extend it.
- It has one protected constructor and 11 commonly listed methods

S. No.	Method	Description
1	getClass()	Returns the runtime class of an object.
2	hashCode()	Returns a hash code value for the object.
3	equals(Object obj)	Compares objects for equality.
4	toString()	Returns a string representation of the object.
5	clone()	Creates and returns a copy of the object, when cloning is supported.
6	finalize()	Historically called before garbage collection; deprecated and should not be used.
7	notify()	Wakes up one thread waiting on the object's monitor.
8	notifyAll()	Wakes up all threads waiting on the object's monitor.
9	wait()	Causes the current thread to wait until notified or interrupted.
10	wait(long timeout)	Waits for notification or until the specified time elapses.
11	wait(long timeout, int nanos)	Waits for notification or until the specified timeout, including nanoseconds.

Example Program:

```
class Student  {
   int id;
   String name;

   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
```

```
    @Override
    public int hashCode() {
        return id;
    }

    @Override
    public boolean equals(Object obj) {
        Student s = (Student) obj;
        return this.id == s.id;
    }

    @Override
    public String toString() {
        return "ID: " + id + ", Name: " + name;
    }
}

public class ObjectClass {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Ravi");
        Student s2 = new Student(101, "Ravi");
        Student s3 = new Student(102, "Raju");

        System.out.println(s1.toString());

        System.out.println("s1 equals s2: " + s1.equals(s2));
        System.out.println("s1 equals s3: " + s1.equals(s3));

        System.out.println("s1 hashCode: " + s1.hashCode());
        System.out.println("s2 hashCode: " + s2.hashCode());
        System.out.println("s3 hashCode: " + s3.hashCode());
    }
}
```

Class Math:

- The Math class is an important class in the java.lang package.
- It provides methods to perform mathematical calculations.
- Some common operations are:
    - Trigonometric functions → sin(), cos(), tan()
    - Power and exponential functions → pow(), exp()
    - Rounding functions → round(), ceil(), floor()
    - Other functions → sqrt(), abs(), max(), min()

- Math is a final class, so we cannot create a subclass of Math.
- All methods in the Math class are static.
- Therefore, we do not need to create an object of the Math class.
- But we use the class name Math to call its methods.
- For example:

```
double x = Math.sqrt(25);
System.out.println(x);
```

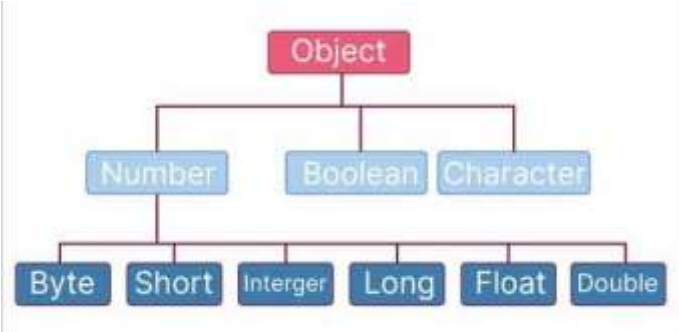
Method	Description	Example
abs(x)	Returns the absolute value of x.	Math.abs(-10) → 10
sqrt(x)	Returns the square root of x.	Math.sqrt(25) → 5.0
pow(x, y)	Returns x raised to the power y.	Math.pow(2, 3) → 8.0
ceil(x)	Returns the smallest integer value greater than or equal to x.	Math.ceil(4.2) → 5.0
floor(x)	Returns the largest integer value less than or equal to x.	Math.floor(4.8) → 4.0
round(x)	Rounds the value to the nearest integer.	Math.round(4.6) → 5
max(x, y)	Returns the greater of x and y.	Math.max(10, 20) → 20
min(x, y)	Returns the smaller of x and y.	Math.min(10, 20) → 10
random()	Returns a random value from 0.0 (inclusive) to 1.0 (exclusive).	Math.random() → 0.735
sin(x)	Returns the sine of x, where x is in radians.	Math.sin(0) → 0.0
cos(x)	Returns the cosine of x, where x is in radians.	Math.cos(0) → 1.0
tan(x)	Returns the tangent of x, where x is in radians.	Math.tan(0) → 0.0

Wrapper Classes

- In Java, everything can be viewed as an object.
- However, primitive data types such as int, char, double, and boolean are not objects.
- Java provides Wrapper Classes to convert primitive data types into objects.
- A wrapper class wraps or encloses a primitive value inside an object.
- Wrapper classes are available in the java.lang package.
- Wrapper class objects are used to pass arguments to collection framework methods.

Primitive Type	Wrapper Class	Example
byte	Byte	Byte b = 10;
short	Short	Short s = 20;
int	Integer	Integer i = 100;
long	Long	Long l = 1000L;
float	Float	Float f = 10.5f;
double	Double	Double d = 25.5;
char	Character	Character c = 'A';
boolean	Boolean	Boolean b = true;

- The eight primitive data types, namely boolean, byte, short, int, long, float, double, and char, are not objects;
- They cannot be passed by references and they are passed by value only.
- In order to provide object representation, eight Wrapper classes are defined in java.lang package.



Constructors of Wrapper Classes (Deprecated) : These constructors are deprecated in favor of using valueOf() or autoboxing.

S. No.	Wrapper Class	Primitive Type	Constructors
1	Byte	byte	Byte(byte value), Byte(String s)
2	Short	short	Short(short value), Short(String s)
3	Integer	int	Integer(int value), Integer(String s)
4	Long	long	Long(long value), Long(String s)
5	Float	float	Float(float value), Float(double value), Float(String s)
6	Double	double	Double(double value), Double(String s)
7	Character	char	Character(char value)
8	Boolean	boolean	Boolean(boolean value), Boolean(String s)

Methods of Wraper Classes:

S. No.	Method	Description
1	static T valueOf(type number)	1. T represents the wrapper class return type. 2. type number represents a primitive value. 3. String number represents a string containing a value.
2	static T valueOf(String number)	
3	public String toString()	Returns the String form of the wrapper object.
4	public static String toString(int i)	Returns the String form of the given primitive.
5	public int intValue()	Returns the value as a primitive int.
6	public byte byteValue()	Returns the value as a primitive byte.
7	public short shortValue()	Returns the value as a primitive short.
8	public long longValue()	Returns the value as a primitive long.
9	public float floatValue()	Returns the value as a primitive float.
10	public double doubleValue()	Returns the value as a primitive doubl

Example 1: Conversion of primitive data types into objects.

```
public class UseofValueOf {
  public static void main(String[] args) {
        Integer a = Integer.valueOf(100);
    Double b = Double.valueOf("12.5");
    Float c = Float.valueOf("5.5");
    Boolean d = Boolean.valueOf("true");
    Long e = Long.valueOf(500L);

    System.out.println(a instanceof Integer);

    System.out.println("Integer Object: " + a);
    System.out.println("Double Object: " + b);
    System.out.println("Float Object: " + c);
    System.out.println("Boolean Object: " + d);
    System.out.println("Long Object: " + e);
  }
}
```

Output:

```
true
Integer Object: 100
Double Object: 12.5
Float Object: 5.5
Boolean Object: true
Long Object: 500
```

Ex 2: Conversion of objects to primitive types.

```
class Ex1 {
   public static void main(String[] args) {

      Integer obj = Integer.valueOf(100);

      byte b = obj.byteValue();
      int i = obj.intValue();
      long l = obj.longValue();
      short s = obj.shortValue();
      double d = obj.doubleValue();
      float f = obj.floatValue();

      System.out.println("Byte value: " + b);
      System.out.println("Int value: " + i);
      System.out.println("Long value: " + l);
      System.out.println("Short value: " + s);
      System.out.println("Double value: " + d);
```

```
            System.out.println("Float value: " + f);

        }
    }
```

Output:

```
Byte value: 100
Int value: 100
Long value: 100
Short value: 100
Double value: 100.0
Float value: 100.0
```

Methods for Conversion of Digital Strings into numbers:

- Parse methods convert a String containing a valid number into its corresponding primitive data type.
- These static methods belong to wrapper classes such as Integer, Double, Float, Byte, Short, and Long.

Method	Wrapper Class	Complete Signature	Converts String to Primitive
parseInt()	Integer	public static int parseInt(String s) throws NumberFormatException	int
parseFloat()	Float	public static float parseFloat(String s) throws NumberFormatException	float
parseDouble()	Double	public static double parseDouble(String s) throws NumberFormatException	double
parseByte()	Byte	public static byte parseByte(String s) throws NumberFormatException	byte
parseShort()	Short	public static short parseShort(String s) throws NumberFormatException	short
parseLong()	Long	public static long parseLong(String s) throws NumberFormatException	long

Example Program:

```
public class Ex2 {
        public static void main(String arg[]) {
                // Parsing various data types
           int intValue = Integer.parseInt("100");
           float floatValue = Float.parseFloat("25.5");
           double doubleValue = Double.parseDouble("1234.5678");
           byte byteValue = Byte.parseByte("20");
           short shortValue = Short.parseShort("3000");
           long longValue = Long.parseLong("999999999");

           // Displaying parsed values
           System.out.println("intValue: " + intValue);
           System.out.println("floatValue: " + floatValue);
           System.out.println("doubleValue: " + doubleValue);
           System.out.println("byteValue: " + byteValue);
           System.out.println("shortValue: " + shortValue);
           System.out.println("longValue: " + longValue);
        }
}
```

O/P:

```
intValue: 100
floatValue: 25.5
doubleValue: 1234.5678
byteValue: 20
shortValue: 3000
longValue: 999999999
```

Auto-boxing and Un-boxing:

- Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper class object.
- Examples:

  int → Integer

  double → Double

  float → Float

  boolean → Boolean

  long → Long

  Ex: int a = 100;

  Integer obj = a;

Auto boxing and unboxing were introduced in Java 5 (JDK 1.5).

- Unboxing is the automatic conversion of a wrapper class object into its corresponding primitive data type.
- Ex:

  Integer obj = 200;

  int b = obj;

Remember:
Autoboxing: Primitive to object.
Unboxing: Object to primitive.

Example Program:

```
public class Auto_Unboxing {
    public static void main(String[] args) {
            int num = 100;

        // Autoboxing
        Integer i = num; //Integer i=Integer.valueOf(num);
        System.out.println("Autoboxed Integer: " + i);

        // Automatic unboxing
        int p = i;
        // Conventional unboxing
        int p1 = i.intValue();

        System.out.println("Auto unboxing val: " + p);
        System.out.println("Conventional unboxing val: " + p1);
    }
}
```

Output:

```
Autoboxed Integer: 100
Auto unboxing val: 100
Conventional unboxing val: 100
```